Lecture 8

- linear models (MSE) $\rightarrow$ DNNs
- convex (quadratic) $\rightarrow$ nonconvex
- using 1st-order (linear) approx.: GD
- using inertia: momentum methods
- using 2nd-order (quadratic) approx.: preconditioned GD

Convergence

For a quadratic loss, ...

$$
\begin{aligned}
L(\theta) &= \frac{1}{2}\theta^\top A\theta + a^\top\theta + b && (A \succ 0; \text{pd}) \\
\delta &= -\eta g && (g = \nabla_\theta L(\theta)) \\
L(\theta+\delta) &= L(\theta) + \nabla_\theta L(\theta)^\top \delta + \frac{1}{2}\delta^\top \frac{\partial^2 L(\theta)}{\partial\theta^2}\delta \\
&\le L(\theta) - \eta\|g\|^2 + \frac{1}{2}\eta^2\lambda_{\max}(A)\|g\|^2 \\
&= L(\theta) - \eta\left(1 - \frac{1}{2}\lambda_{\max}(A)\eta\right)\|g\|^2
\end{aligned}
$$

Theorem

If we run GD on a quad. loss with $\eta < \frac{2}{\lambda_{\max}(A)}$ and $A = \frac{\partial^2 L}{\partial\theta^2} \succ 0$, then the loss decreases.[23]

[23] Actually, we have global convergence iff $\eta < \frac{2}{\lambda_{\max}(A)}$.

Convex Set

A set X is said to be convex set if

$$x, y \in X \rightarrow \lambda x + (1 - \lambda) y \in X, \ \forall \lambda \in [0, 1].$$

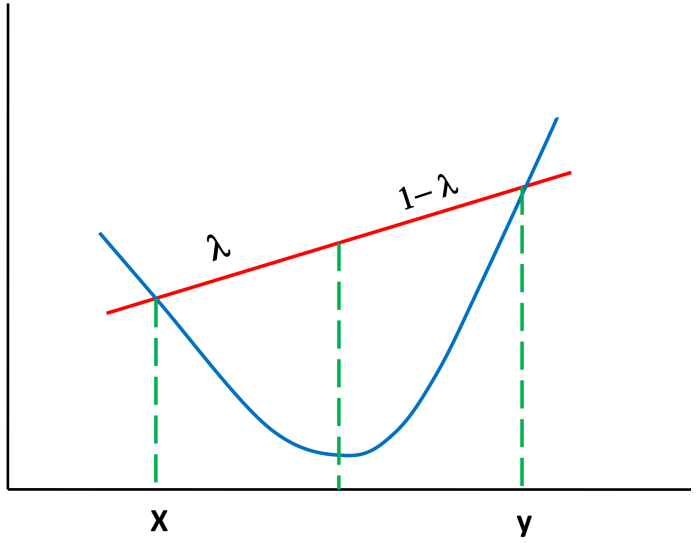
Convex Function

A function $f : X \to \mathbb{R}$ is said to be convex if X is convex and

$$f(\lambda x + (1-\lambda)y) \leq \lambda f(x) + (1-\lambda)f(y)$$

for any $x, y \in X$ and for any $\lambda \in [0, 1]$.

- convex ($\leq$), strictly convex ($<$), concave ($\geq$)
- A function g is concave iff $-g$ is convex.



- If f is differentiable and convex, then the following holds

$$f(x) \geq \underbrace{f(y) + \nabla f(y)(x-y)}_{\text{linear approx. at } y}$$

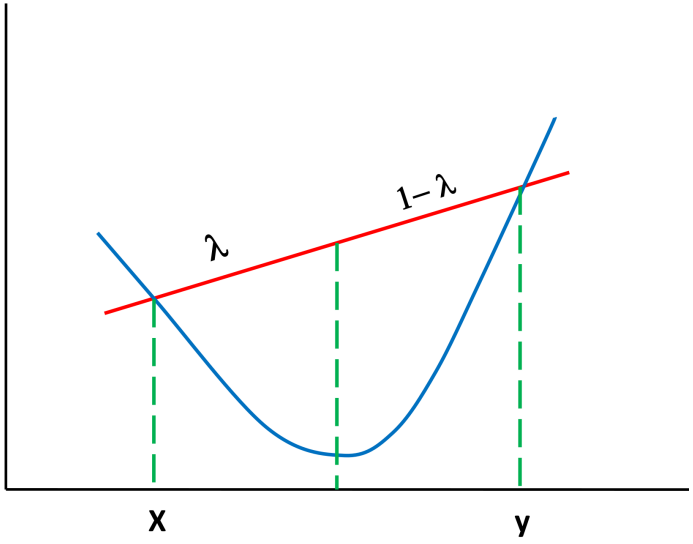
  for any $x, y \in X$.
- If f is differentiable, then f is convex iff, for any $x, y \in X$,

$$\langle \nabla f(x) - \nabla f(y), x - y \rangle \geq 0$$

- If f is twice differentiable, then f is convex [strictly convex] iff

$$\nabla_x^2 f(x) \succeq 0 [\nabla_x^2 f(x) \succ 0]$$

  for any $x \in X$.



Theorem (Jensen's Inequality)

For a convex function $f : X \to \mathbb{R}$, we have

$$f(\sum_i \lambda_i x_i) \leq \sum_i \lambda_i f(x_i)$$

for any $x_i \in X$ and for any $\lambda_i \in [0, 1]$ such that $\sum_i \lambda_i = 1$. Similarly, we have[24]

$$f(\mathbb{E}_X[X]) \leq \mathbb{E}_X[f(X)]$$

[24] think of λ_i as a pdf $p_X(x)$.

Quiz

- Show that the KL-divergence is always nonnegative.
- A function $f(x) = x^\top Ax$ is convex if A is a psd matrix.

Convex

If f is differentiable, then f is convex iff, for any $x, y \in X$,

$$\langle \nabla f(x) - \nabla f(y), x - y \rangle \geq 0.$$

(m-)Strongly Convex (not too flat)

A function $f : X \to \mathbb{R}$ is said to be m-strongly convex ($m > 0$) if

$$\langle \nabla f(x) - \nabla f(y), x - y \rangle \geq m\|x - y\|^2$$

for any $x, y \in X$ where X is a normed space.

- strongly convex $\to$ strictly convex $\to$ convex (= "0-strongly convex")
- If f is twice differentiable, then f is strongly convex iff $\nabla_x^2 f(x) \succeq mI$

(m-)Strongly Convex (not too flat)

A function $f : X \to \mathbb{R}$ is said to be m-strongly convex ($m > 0$) if

$$\langle \nabla f(x) - \nabla f(y), x - y \rangle \geq m\|x - y\|^2,$$

or equivalently, if

$$f(x) - \underbrace{\Big(f(y) + \nabla f(y)^\top (x - y)\Big)}_{\text{linear approx. at } y} \geq \frac{m}{2}\|x - y\|^2$$

for any $x, y \in X$ where X is a normed space.

- strongly convex $\to$ strictly convex $\to$ convex (="0-strongly convex")
- If f is twice differentiable, then f is strongly convex iff $\nabla_x^2 f(x) \succeq mI$.

Lipschitz Continuity (not too steep)

A function $f : X \to \mathbb{R}$ is L-Lipschitz continuous (L: Lipschitz constant) if, for any $x, y \in X$,

$$|f(x) - \underbrace{f(y)}_{\text{value at } y}| \leq L\|x - y\|.$$

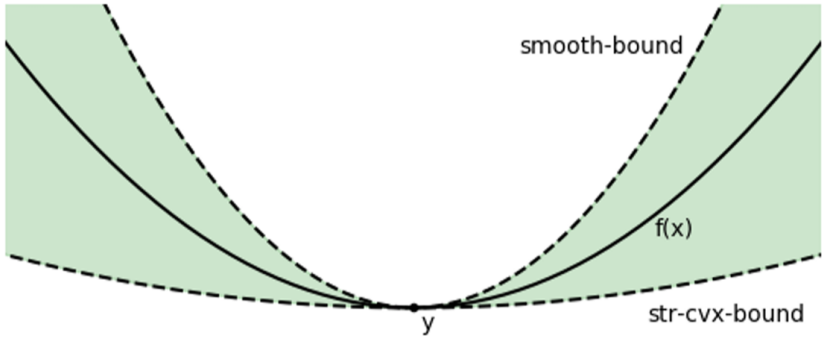
β-smoothness (not too sharp)

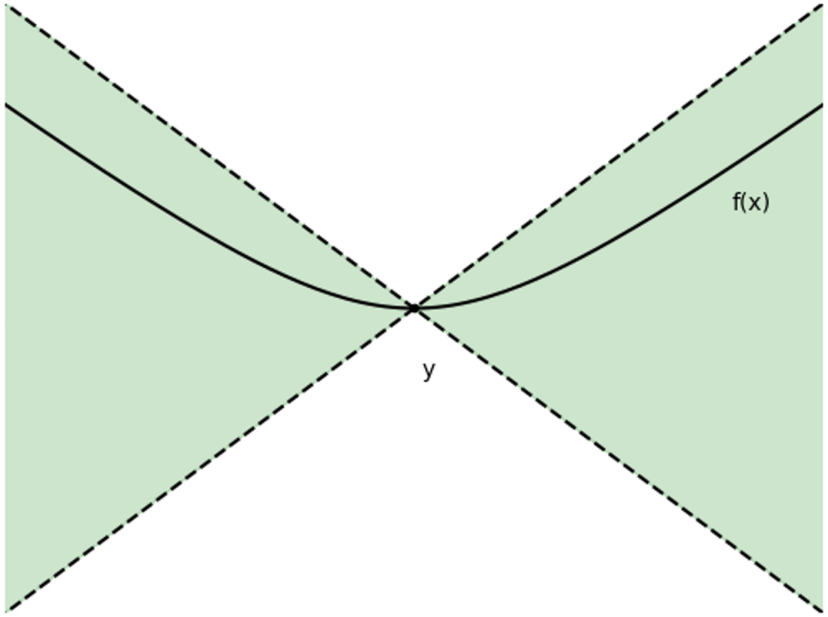
A function $f : X \to \mathbb{R}$ is β-smooth for some constant $\beta > 0$ if

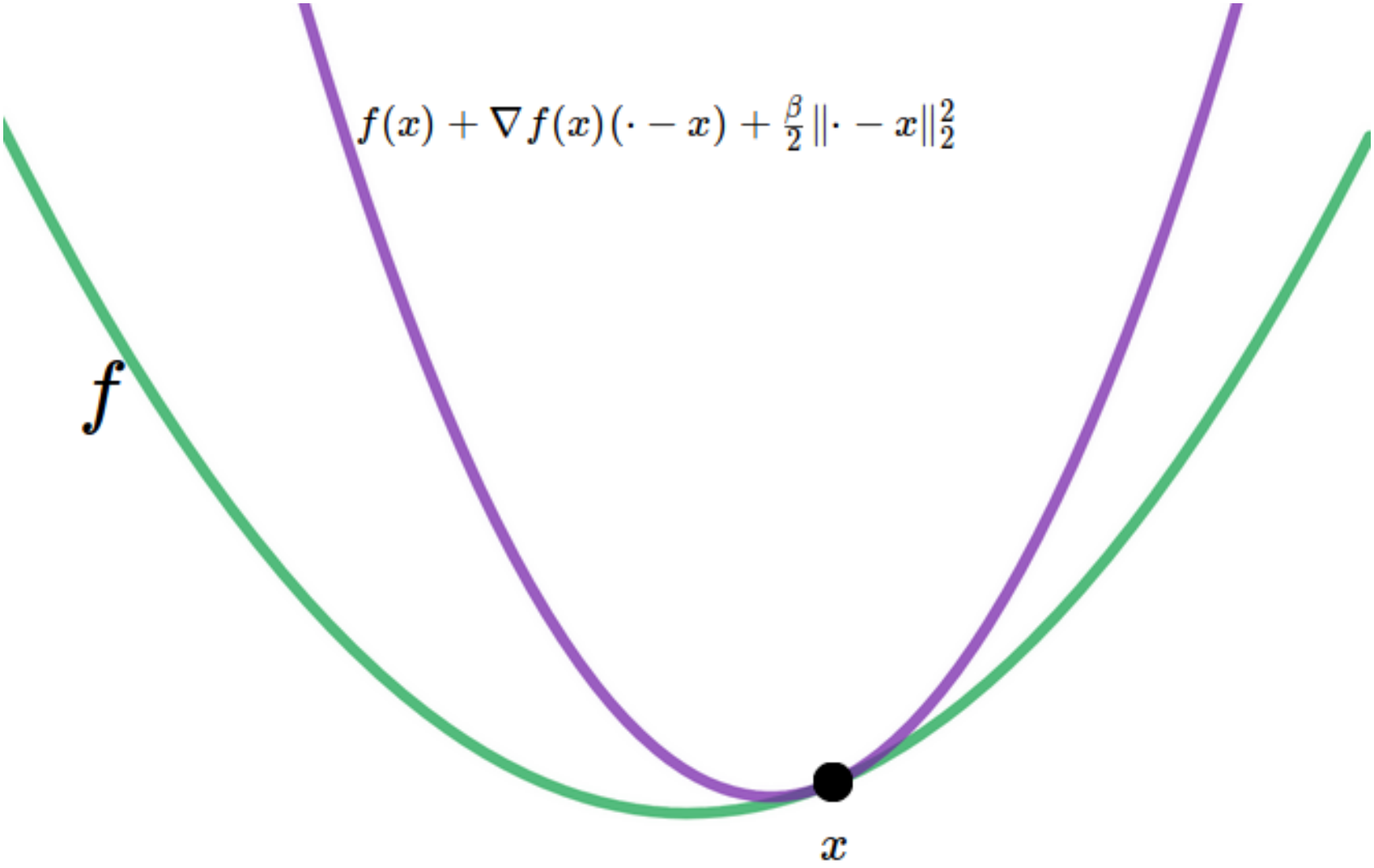
$$|f(x) - \underbrace{\Big(f(y) + \nabla f(y)^\top (x - y)\Big)}_{\text{linear approx. at } y}| \leq \frac{\beta}{2}\|x - y\|^2$$

for any $x, y \in X$ where X is a normed space.

- f is β-smooth iff ∇f is β-Lipschitz continuous.
- If f is twice differentiable, then f is β-smooth iff $\nabla_x^2 f(x) \preceq \beta I$


smooth-bound
f(x)
y
str-cvx-bound


f(x)
y


$f(x) + \nabla f(x)(\cdot - x) + \frac{\beta}{2}\|\cdot - x\|_2^2$
f
x

Polyak-Łojasiewicz condition (grad. not too small)

A function $f : X \to \mathbb{R}$ satisfies c-PŁ-condition if

$$\frac{1}{2}\|\nabla f(x)\|^2 \geq c(f(x) - f(x^*))$$

for any $x \in X$ where $f(x^*)$ is the optimal function value.

Linear Convergence

For a loss with β-smoothness (not too sharp) and c-PŁ-condition (grad. not too small), ...

$$
\begin{aligned}
L_{t+1} - L_t &\leq -\eta \|\nabla L\|^2 + \frac{\eta^2}{2}\beta\|\nabla L\|^2 && (\beta\text{-smoothness}) \\
&= -\eta\|\nabla L\|^2\left(1 - \frac{\eta\beta}{2}\right) \\
&= -\frac{1}{2\beta}\|\nabla L\|^2 && (\eta = \frac{1}{\beta}) \\
&\leq -\frac{c}{\beta}(L_t - L^*) && (c\text{-PŁ-condition})
\end{aligned}
$$

$$
L_{t+1} - L_t + (L_t - L^*) \leq (L_t - L^*) - \frac{c}{\beta}(L_t - L^*)
$$

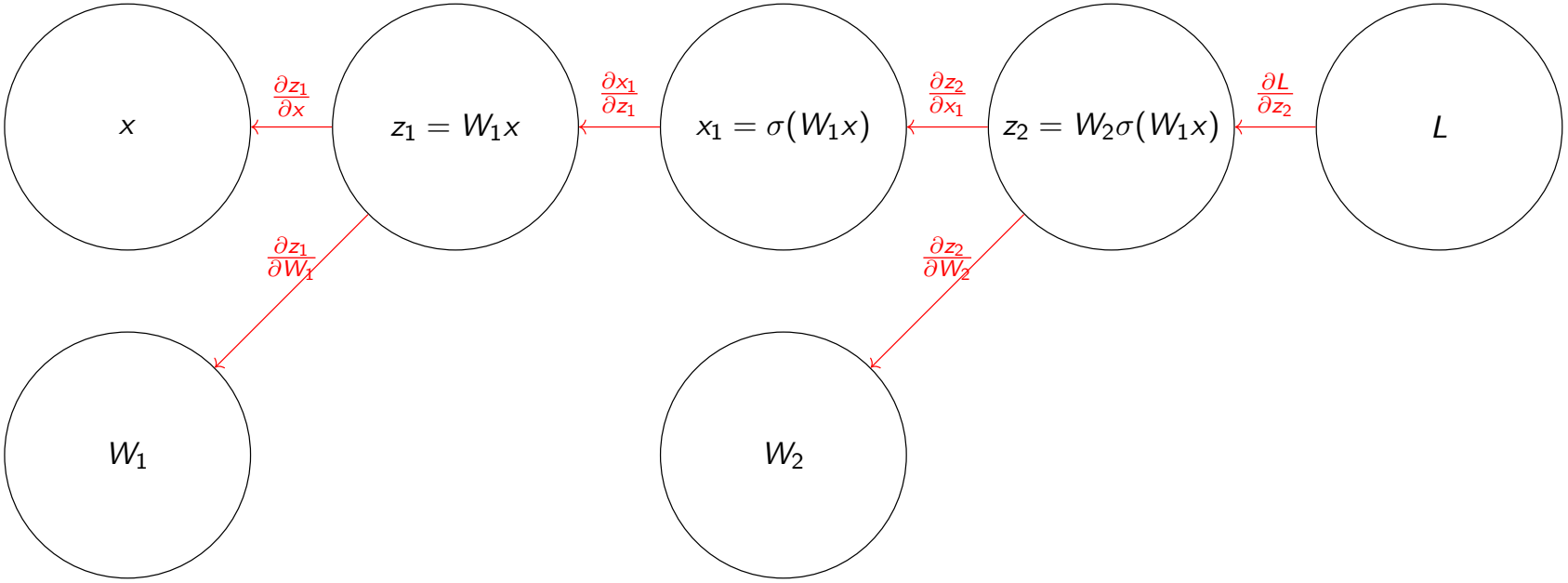
$$
\begin{aligned}
L_{t+1} - L^* &\leq \left(1 - \frac{c}{\beta}\right)(L_t - L^*) \\
L_t - L^* &\leq \left(1 - \frac{c}{\beta}\right)^t (L_0 - L^*) && (L_t \in O(e^{-c't}))
\end{aligned}
$$

- second-order methods
- Newton's method
- quasi-Newton methods, (L-)BFGS
- trust-region methods
- learning rate schedule
- iterate averaging

SGD can often find surprisingly "good" solutions even though the DNN loss is not a convex objective.

Introduction
Neural Networks
Optimization
Neural Networks + Optimization
Generalization
etc.

Backpropagation[25] - Computation Graph (DAG[26])



$$\underbrace{\frac{\partial L}{\partial W_1}}_{1\times d(W_1)} = \overbrace{\underbrace{\frac{\partial L}{\partial z_2}}_{1\times d(z_2)} \underbrace{\frac{\partial z_2}{\partial x_1}}_{d(z_2)\times d(x_1)}}^{\text{vector-Jacobian product}(\rightarrow)} \underbrace{\frac{\partial x_1}{\partial z_1}}_{d(x_1)\times d(z_1)} \underbrace{\frac{\partial z_1}{\partial W_1}}_{d(z_1)\times d(W_1)}$$

[25] automatic differentiation (autodiff)

[26] Directed Acyclic Graph

Backpropagation

Q. What is the time complexity of backpropagation (matmul)?
(assume $d(z_2) = d(x_1) = d(z_1) = d(W_1) = n$)

$$\frac{\partial p_y}{\partial z} = p_y(e_y - p)^\top \qquad (p = \mathrm{softmax}(z))$$

$$\frac{\partial p}{\partial z} = \mathrm{diag}(p) - pp^\top$$

$$\text{CE: } \frac{\partial L}{\partial z} = \frac{\partial[-\log(p_y)]}{\partial z} = (p - e_y)^\top \qquad (L(z, y) = -\log p_y(z))$$

$$\text{MSE: } \frac{\partial L}{\partial z} = (z - z_0)^\top \qquad (L(z, z_0) = \frac{1}{2}\|z - z_0\|^2)$$

$$\frac{\partial \mathrm{ReLU}(z)}{\partial z} = \mathrm{diag}(\mathbf{1}(z > 0))^{27}$$

$$\frac{\partial Wx}{\partial x} = W$$

[27] undefined for the elements with 0

An Example - Pytorch

```
[1]  import torch
     ✓ 1.4s

[2]  x1 = torch.tensor([1.], requires_grad=True)
     x2 = torch.tensor([2.], requires_grad=True)
     x3 = torch.tensor([3.], requires_grad=True)
     ✓ 0.0s

[3]  y = x1*x2*x3
     ✓ 0.0s

[4]  y.backward()
     ✓ 1.0s

[5]  print(x1.grad)
     ✓ 0.0s

...  tensor([6.])
```

Hyperparameter Tuning

SGD; learning rate $\eta > 0$ and mini-batch $B \sim \mathcal{P}^b$ $(|B| = b)$

$$\theta \leftarrow \theta - \eta \nabla_\theta L_B(\theta)$$

Exploding Gradient

gradient clipping

$$g' = \min\left(1, \frac{c}{\|g\|}\right) g$$

the norm of g' cannot exceed c.

Vanishing Gradient

- non-saturating activation functions (e.g., ReLU)
- residual connection
- normalization layer
- careful parameter initialization

Residual Connection - ResNet [HZR+16]

$$y = F(x) + x$$

where $F(x)$ and x have the same shape.

$$\frac{\partial y}{\partial x} = \frac{\partial F(x)}{\partial x} + I$$

Batch Normalization $(\{x^{(i)}\}_{i=1}^m \mapsto \{y^{(i)}\}_{i=1}^m)$

$$B = \{x^{(1)}, x^{(2)}, \cdots, x^{(m)}\} \qquad (x^{(i)} \in \mathbb{R}^d)$$

$$(\mu_B)_k = \frac{1}{m}\sum_{i=1}^m x_k^{(i)} \in \mathbb{R}$$

$$(\sigma_B)_k^2 = \frac{1}{m}\sum_{i=1}^m (x_k^{(i)} - (\mu_B)_k)^2 \in \mathbb{R}$$

$$\hat{x}_k^{(i)} = \frac{x_k^{(i)} - (\mu_B)_k}{\sqrt{(\sigma_B)_k^2 + \epsilon}} \in \mathbb{R} \qquad (\epsilon = \text{ 1e-5 for stability})$$

$$\hat{x}^{(i)} = \frac{x^{(i)} - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} \in \mathbb{R}^d \qquad (\text{vector notation})$$

$$y^{(i)} = \gamma \odot \hat{x}^{(i)} + \beta \qquad (\gamma, \beta : \text{ learned})$$

where $\mu_B, \sigma_B^2, \gamma, \beta$ are all vector.

Batch Normalization

During the training stage, we track the (population) running statistics using exponential moving average (EMA) with a momentum (default $\alpha = 0.1$).

$$\mu_{B_t} = \frac{1}{m} \sum_{i \in B_t} x^{(i)} \in \mathbb{R}^d$$

$$\sigma^2_{B_t} = \frac{1}{m} \sum_{i \in B_t} (x^{(i)} - \mu_{B_t})^2 \in \mathbb{R}^d \qquad \text{(vector notation)}$$

$$\text{EMA}_0 = [\mu_{B_0}, \frac{m}{m-1}\sigma^2_{B_0}] \qquad (\frac{m}{m-1} : \text{ unbiased est.})$$

$$\text{EMA}_t = \alpha[\mu_{B_t}, \frac{m}{m-1}\sigma^2_{B_t}] + (1-\alpha)\text{EMA}_{t-1}$$

pytorch/torch/nn/modules/batchnorm.py
pytorch/torch/nn/functional.py
pytorch/aten/src/ATen/native/Normalization.cpp

BN vs. LN (Layer Normalization)

- BN: across the batch axis (N) & the spatial axes (H,W)
- LN: across the channel axis (C) & the spatial axes (H,W)

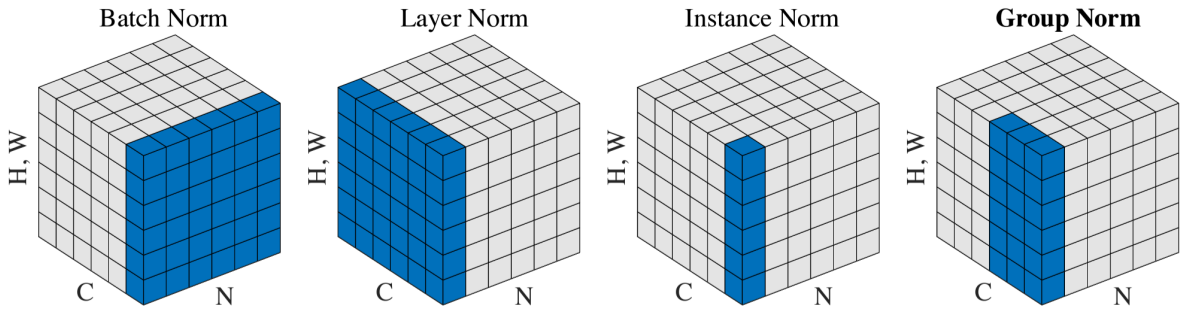


Figure 2. **Normalization methods**. Each subplot shows a feature map tensor, with N as the batch axis, C as the channel axis, and (H, W) as the spatial axes. The pixels in blue are normalized by the same mean and variance, computed by aggregating the values of these pixels.

Figure: Group Normalization

Why does BN work?

The exact reasons for the benefits of BN are still unclear. BN ...

- reduces "internal covariate shift" (?) [IS15]
  - Training DNNs is complicated because the distribution of each layer's inputs (internal covariate) changes during training, as the parameters of the previous layer changes.
- makes the optimization landscape significantly smoother [STI+18]
- reduces the sensitivity to the learning rate
- acts like a regularizer

Parameter Initialization

Sampling parameters from a standard normal with fixed variance can result in exploding activations or gradients. We want to keep the variance of the gradients from blowing up.

$$w_{ij} \sim \mathcal{N}(0, \sigma^2)$$

$$\sigma^2 = \frac{1}{n_{in}} \qquad \text{(Lecun)}$$

$$\sigma^2 = \frac{2}{n_{in} + n_{out}} \qquad \text{(Xavier/Glorot)}$$

$$\sigma^2 = \frac{2}{n_{in}} \qquad \text{(He)}$$

where n_{in}/n_{out} is the fan-in/-out of a unit (# of incoming/outgoing connections).